

Plan de Autenticación Avanzada TeacherBot v2.0

Módulo de Autenticación por Perfiles RBAC
Unidad Educativa Oxford

Arquitectura: JWT Stateless + httpOnly Cookies + Refresh Token Rotation

Roles: admin · vicerrector · docente · estudiante · sistemas

Agosto 2026

Plan de Autenticación Avanzada — TeacherBot

Unidad Educativa Oxford | Agosto 2026

1. DIAGNÓSTICO ACTUAL

Estado: SIN autenticación

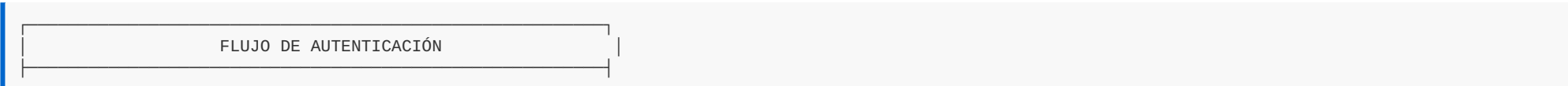
Componente	Estado
Backend (<code>serve.cjs</code>)	Cero middleware de auth
Frontend (React SPA)	Rutas abiertas: <code>/</code> , <code>/docente</code> , <code>/vicerrectorado</code> , <code>/sistemas</code>
API REST	Todos los endpoints son públicos
Base de datos	Tabla <code>users</code> existe con roles pero nunca se usa para login
Landing page	Links directos a <code>/docente</code> , <code>/vicerrectorado</code> , <code>/sistemas</code> sin protección

Riesgos

- Cualquiera accede al panel de sistemas y configura proveedores IA
- Datos de estudiantes expuestos
- KPIs y notas visibles sin restricción
- Sin trazabilidad de quién hace qué

2. ARQUITECTURA PROPUESTA

2.1 Método: JWT Stateless + httpOnly Cookies



```
1. POST /api/auth/login {email, password}
  ↓
2. Backend valida credenciales contra SQLite
  ↓
3. Genera JWT {userId, role, name} + refreshToken
  ↓
4. Set-Cookie: token (httpOnly, Secure, SameSite=Lax)
  ↓
5. Cliente recibe sesión y user info
  ↓
6. Cada request incluye Cookie automáticamente
  ↓
7. Middleware parseJWT() valida token en cada endpoint
  ↓
8. Si expira → POST /api/auth/refresh (refresh cookie)
```

2.2 Elección: ¿Por qué JWT + httpOnly Cookies y no sesiones?

Opción	Ventaja	Desventaja
JWT + Cookies	Stateless, sin DB para sesiones, escalable	Revocación más compleja
Server Sessions	Revocación instantánea	Necesita DB/Redis, stateful
LocalStorage JWT	Simple	Vulnerable a XSS

Decisión: JWT httpOnly cookies — El backend ya es un monolito sin Redis. La cookie httpOnly previene XSS. El refresh token permite revocación.

3. BASE DE DATOS: MIGRACIONES

3.1 Tabla `users` actual → Extender

```
-- Añadir columnas de autenticación
ALTER TABLE users ADD COLUMN password_hash TEXT;
ALTER TABLE users ADD COLUMN last_login TEXT;
ALTER TABLE users ADD COLUMN failed_attempts INTEGER DEFAULT 0;
ALTER TABLE users ADD COLUMN locked_until TEXT;
ALTER TABLE users ADD COLUMN must_change_password INTEGER DEFAULT 0;
ALTER TABLE users ADD COLUMN created_by TEXT REFERENCES users(id);
```

3.2 Nueva tabla: `refresh_tokens`

```
CREATE TABLE refresh_tokens (  
  id TEXT PRIMARY KEY,  
  user_id TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  token_hash TEXT NOT NULL UNIQUE,  
  expires_at TEXT NOT NULL,  
  created_at TEXT NOT NULL DEFAULT (datetime('now')),  
  revoked INTEGER NOT NULL DEFAULT 0  
);  
CREATE INDEX idx_refresh_user ON refresh_tokens(user_id);
```

3.3 Nueva tabla: `audit_log`

```
CREATE TABLE audit_log (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  user_id TEXT REFERENCES users(id),  
  action TEXT NOT NULL,  
  entity TEXT,  
  entity_id TEXT,  
  ip_address TEXT,  
  user_agent TEXT,  
  details TEXT,  
  created_at TEXT NOT NULL DEFAULT (datetime('now'))  
);  
CREATE INDEX idx_audit_user ON audit_log(user_id);  
CREATE INDEX idx_audit_action ON audit_log(action);  
CREATE INDEX idx_audit_date ON audit_log(created_at);
```

3.4 Semilla de usuarios con passwords

```
-- bcrypt hash de "Oxford2026!" para cada usuario inicial  
UPDATE users SET password_hash = '$2b$10$...' WHERE id = 'u9'; -- admin  
UPDATE users SET password_hash = '$2b$10$...' WHERE id = 'u1'; -- docente María  
-- ... etc
```

4. PERFILES DE USUARIO (RBAC)

4.1 Definición de Roles

Rol	Permisos	Rutas	API
admin	Todo	/admin/*	CRUD total
vicerrector	KPIs, reportes, supervisión	/vicerrectorado/*	GET: cursos, entregas, KPIs, usuarios
docente	Sus cursos, tareas, evaluación	/docente/*	GET/POST: cursos propios, tareas, evaluar IA
estudiante	Ver sus entregas y feedback	/estudiante/*	GET: mis entregas, notas, feedback
sistemas	Monitoreo, IA config, logs	/sistemas/*	GET/PUT: config IA, proveedores, sync

4.2 Matriz de Permisos Detallada

	admin	vicerrector	docente	estudiante	sistemas
Ver landing page	✓	✓	✓	✓	✓
Login	✓	✓	✓	✓	✓
Mi perfil	✓	✓	✓	✓	✓
CURSOS					
Listar todos	✓	✓	-	-	-
Listar míos	-	-	✓	✓	-
Ver detalle	✓	✓	✓(suyo)	✓(suyo)	-
Crear/editar	✓	-	-	-	-
TAREAS					
Listar todas	✓	✓	-	-	-
Listar mías	-	-	✓(suyas)	✓(suyas)	-
Evaluar con IA	✓	-	✓(suyas)	-	-
Ver rúbricas	✓	✓	✓(suyas)	✓(suyas)	-
ENTREGAS					
Listar todas	✓	✓	-	-	-
Listar por curso	✓	✓	✓(suyas)	-	-
Ver feedback IA	✓	✓	✓(suyas)	✓(propias)	-
Ajustar nota	✓	-	✓(suyas)	-	-
USUARIOS					
Listar	✓	✓	-	-	✓
Crear/editar	✓	-	-	-	-

Reset password	✓	-	-	-	-
IA CONFIG					
Ver	✓	-	-	-	✓
Editar	✓	-	-	-	✓
Test proveedor	✓	-	-	-	✓
SISTEMA					
Ver logs	✓	-	-	-	✓
Ejecutar sync	✓	-	-	-	✓
Monitoreo KPIs	✓	✓	-	-	✓
AUDITORÍA					
Ver logs auditoría	✓	✓	-	-	-

5. BACKEND: IMPLEMENTACIÓN (`serve.cjs`)

5.1 Nuevas dependencias

```
pnpm add bcryptjs jsonwebtoken uuid
```

5.2 Módulo de autenticación (añadir al inicio de `serve.cjs`)

```
const crypto = require('crypto');
const jwt = require('jsonwebtoken');
const bcrypt = require('bcryptjs');

// — Config —————
const JWT_SECRET = process.env.JWT_SECRET || crypto.randomBytes(64).toString('hex');
const ACCESS_TOKEN_EXPIRY = '15m'; // 15 minutos
const REFRESH_TOKEN_EXPIRY = '7d'; // 7 días
const MAX_FAILED_ATTEMPTS = 5;
const LOCK_DURATION_MINUTES = 30;
const BCRYPT_ROUNDS = 12;

// — Auth Helpers —————

function hashPassword(plain) {
  return bcrypt.hashSync(plain, BCRYPT_ROUNDS);
}

function verifyPassword(plain, hash) {
  return bcrypt.compareSync(plain, hash);
}
```

```

function generateTokens(user) {
  const accessToken = jwt.sign(
    { sub: user.id, role: user.role, name: user.name },
    JWT_SECRET,
    { expiresIn: ACCESS_TOKEN_EXPIRY }
  );

  // Refresh token: opaque random string, stored hashed in DB
  const refreshToken = crypto.randomBytes(48).toString('hex');
  const refreshTokenHash = crypto.createHash('sha256').update(refreshToken).digest('hex');

  db.prepare(`INSERT INTO refresh_tokens (id, user_id, token_hash, expires_at)
    VALUES (?, ?, ?, datetime('now', '+7 days'))`)
    .run(crypto.randomUUID(), user.id, refreshTokenHash);

  return { accessToken, refreshToken };
}

function verifyAccessToken(token) {
  try {
    return jwt.verify(token, JWT_SECRET);
  } catch(e) {
    return null;
  }
}

// — Auth Middleware —————

function parseAuthCookie(req) {
  const cookie = req.headers.cookie || '';
  const match = cookie.match(/token=([^\;]+)/);
  return match ? match[1] : null;
}

function requireAuth(req, res) {
  const token = parseAuthCookie(req);
  if (!token) {
    res.writeHead(401, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify({ error: 'No autenticado' }));
    return null;
  }

  const payload = verifyAccessToken(token);
  if (!payload) {
    res.writeHead(401, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify({ error: 'Token expirado o inválido' }));
    return null;
  }

  return payload; // { sub, role, name, iat, exp }
}

```

```

}

function requireRole(req, res, allowedRoles) {
  const user = requireAuth(req, res);
  if (!user) return null;

  if (!allowedRoles.includes(user.role)) {
    res.writeHead(403, {'Content-Type': 'application/json'});
    res.end(JSON.stringify({ error: 'No autorizado para este recurso' }));
    return null;
  }

  return user;
}

// Auditoría
function auditLog(userId, action, entity, entityId, req) {
  db.prepare(`INSERT INTO audit_log (user_id, action, entity, entity_id, ip_address, user_agent)
VALUES (?, ?, ?, ?, ?, ?)`)
    .run(userId, action, entity, entityId,
      req.headers['x-forwarded-for'] || req.socket.remoteAddress || '',
      req.headers['user-agent'] || '');
}

```

5.3 Nuevos Endpoints de Autenticación

```

// — POST /api/auth/login —————
if (apiPath === '/auth/login' && req.method === 'POST') {
  return readBody(req).then(body => {
    const { email, password } = body;
    if (!email || !password) return error(res, 'Email y contraseña requeridos', 400);

    const user = db.prepare('SELECT * FROM users WHERE email = ? AND active = 1').get(email);
    if (!user) return error(res, 'Credenciales inválidas', 401);

    // Check lockout
    if (user.locked_until && new Date(user.locked_until) > new Date()) {
      return error(res, 'Cuenta bloqueada temporalmente. Intente en 30 minutos.', 423);
    }

    if (!user.password_hash) return error(res, 'Cuenta no configurada para login', 401);

    if (!verifyPassword(password, user.password_hash)) {
      const attempts = (user.failed_attempts || 0) + 1;
      if (attempts >= MAX_FAILED_ATTEMPTS) {
        db.prepare('UPDATE users SET failed_attempts = ?, locked_until = datetime('now', '+30 minutes') WHERE id = ?')
          .run(attempts, user.id);
      } else {
        db.prepare('UPDATE users SET failed_attempts = ? WHERE id = ?').run(attempts, user.id);
      }
    }
  });
}

```

```

    }
    return error(res, 'Credenciales inválidas', 401);
  }

  // Success
  db.prepare('UPDATE users SET failed_attempts = 0, locked_until = NULL, last_login = datetime(\'now\') WHERE id = ?')
    .run(user.id);

  const { accessToken, refreshToken } = generateTokens(user);

  auditLog(user.id, 'login', 'user', user.id, req);

  // Set cookies
  const isProduction = true; // Always secure in production
  res.writeHead(200, {
    'Content-Type': 'application/json',
    'Set-Cookie': [
      `token=${accessToken}; HttpOnly; Secure; SameSite=Lax; Path=/; Max-Age=900`,
      `refreshToken=${refreshToken}; HttpOnly; Secure; SameSite=Lax; Path=/api/auth; Max-Age=604800`
    ]
  });

  res.end(JSON.stringify({
    user: {
      id: user.id,
      name: user.name,
      email: user.email,
      role: user.role,
      avatar_url: user.avatar_url
    }
  }));
}

// — POST /api/auth/refresh —————
if (apiPath === '/auth/refresh' && req.method === 'POST') {
  const cookie = req.headers.cookie || '';
  const match = cookie.match(/refreshToken=([^\;]+)/);
  if (!match) return error(res, 'No refresh token', 401);

  const refreshToken = match[1];
  const refreshHash = crypto.createHash('sha256').update(refreshToken).digest('hex');

  const stored = db.prepare(`SELECT * FROM refresh_tokens
    WHERE token_hash = ? AND revoked = 0 AND expires_at > datetime('now')`)
    .get(refreshHash);

  if (!stored) return error(res, 'Refresh token inválido o expirado', 401);

  const user = db.prepare('SELECT * FROM users WHERE id = ? AND active = 1').get(stored.user_id);
  if (!user) return error(res, 'Usuario no encontrado', 401);
}

```

```

// Rotate refresh token (revoke old, issue new)
db.prepare('UPDATE refresh_tokens SET revoked = 1 WHERE id = ?').run(stored.id);

const tokens = generateTokens(user);

res.writeHead(200, {
  'Content-Type': 'application/json',
  'Set-Cookie': [
    `token=${tokens.accessToken}; HttpOnly; Secure; SameSite=Lax; Path=/; Max-Age=900`,
    `refreshToken=${tokens.refreshToken}; HttpOnly; Secure; SameSite=Lax; Path=/api/auth; Max-Age=604800`
  ]
});

res.end(JSON.stringify({ ok: true }));
}

// — POST /api/auth/logout —————
if (apiPath === '/auth/logout' && req.method === 'POST') {
  const cookie = req.headers.cookie || '';
  const refreshMatch = cookie.match(/refreshToken=([^;]+)/);
  if (refreshMatch) {
    const refreshHash = crypto.createHash('sha256').update(refreshMatch[1]).digest('hex');
    db.prepare('UPDATE refresh_tokens SET revoked = 1 WHERE token_hash = ?').run(refreshHash);
  }

  res.writeHead(200, {
    'Content-Type': 'application/json',
    'Set-Cookie': [
      'token=; HttpOnly; Secure; SameSite=Lax; Path=/; Max-Age=0',
      'refreshToken=; HttpOnly; Secure; SameSite=Lax; Path=/api/auth; Max-Age=0'
    ]
  });

  res.end(JSON.stringify({ ok: true }));
}

// — GET /api/auth/me —————
if (apiPath === '/auth/me' && req.method === 'GET') {
  const user = requireAuth(req, res);
  if (!user) return;

  const full = db.prepare('SELECT id, name, email, role, avatar_url, last_login FROM users WHERE id = ?').get(user.sub);
  if (!full) return error(res, 'Usuario no encontrado', 404);

  return json(res, full);
}

// — PUT /api/auth/password —————
if (apiPath === '/auth/password' && req.method === 'PUT') {
  const user = requireAuth(req, res);

```

```

if (!user) return;

return readBody(req).then(body => {
  const { currentPassword, newPassword } = body;
  if (!currentPassword || !newPassword) return error(res, 'Contraseñas requeridas', 400);
  if (newPassword.length < 8) return error(res, 'Mínimo 8 caracteres', 400);

  const u = db.prepare('SELECT password_hash FROM users WHERE id = ?').get(user.sub);
  if (!verifyPassword(currentPassword, u.password_hash)) {
    return error(res, 'Contraseña actual incorrecta', 401);
  }

  db.prepare('UPDATE users SET password_hash = ?, must_change_password = 0 WHERE id = ?')
    .run(hashPassword(newPassword), user.sub);

  auditLog(user.sub, 'change_password', 'user', user.sub, req);

  return json(res, { ok: true });
});
}

```

5.4 Proteger Endpoints Existentes

```

// EJEMPLO: Proteger GET /api/courses según rol
if (apiPath === '/courses' && req.method === 'GET') {
  const user = requireAuth(req, res);
  if (!user) return;

  let courses;
  if (user.role === 'docente') {
    // Solo sus cursos
    courses = db.prepare('SELECT * FROM courses WHERE teacher_id = ?').all(user.sub);
  } else if (user.role === 'estudiante') {
    // Solo cursos donde está matriculado
    courses = db.prepare(`
      SELECT c.* FROM courses c
      JOIN enrollments e ON e.course_id = c.id
      WHERE e.student_id = ?
    `).all(user.sub);
  } else if (['admin', 'vicerrector'].includes(user.role)) {
    // Todos los cursos
    courses = db.prepare('SELECT * FROM courses').all();
  } else {
    return error(res, 'No autorizado', 403);
  }

  return json(res, courses);
}

```

```
// EJEMPLO: GET /api/ai/config – solo admin y sistemas
if (apiPath === '/ai/config' && req.method === 'GET') {
  const user = requireRole(req, res, ['admin', 'sistemas']);
  if (!user) return;
  // ... código existente
}

// EJEMPLO: POST /api/ai/evaluate – solo admin y docente (sus tareas)
if (apiPath === '/ai/evaluate' && req.method === 'POST') {
  const user = requireRole(req, res, ['admin', 'docente']);
  if (!user) return;

  // Validar que el docente solo evalúe entregas de SUS cursos
  if (user.role === 'docente') {
    // ... validación adicional
  }
  // ... código existente
}
```

5.5 Mapeo completo de protección por endpoint

ENDPOINT	ADMIN	VICE	DOCEN	ESTUD	SIST
POST /api/auth/login	✓	✓	✓	✓	✓
POST /api/auth/refresh	✓	✓	✓	✓	✓
POST /api/auth/logout	✓	✓	✓	✓	✓
GET /api/auth/me	✓	✓	✓	✓	✓
PUT /api/auth/password	✓	✓	✓	✓	✓
GET /api/courses	✓	✓	✓*	✓*	-
GET /api/assignments	✓	✓	✓*	✓*	-
GET /api/submissions	✓	✓	✓*	✓**	-
GET /api/students	✓	✓	✓*	-	-
GET /api/teachers	✓	✓	-	-	✓
GET /api/stats	✓	✓	-	-	-
GET /api/kpis	✓	✓	-	-	-
GET /api/logs	✓	-	-	-	✓
GET /api/health	-	-	-	-	-
POST /api/sync	✓	-	-	-	✓
POST /api/ai/evaluate	✓	-	✓***	-	-
POST /api/ai/evaluate-batch	✓	-	✓***	-	-
GET /api/ai/evaluate-stream/:id	✓	-	✓***	-	-
GET /api/ai/status/:id	✓	✓	✓*	✓**	-
GET /api/ai/config	✓	-	-	-	✓
PUT /api/ai/config	✓	-	-	-	✓

GET	/api/ai/providers	✓	-	-	-	✓
POST	/api/ai/providers	✓	-	-	-	-
PUT	/api/ai/providers/:id	✓	-	-	-	-
DEL	/api/ai/providers/:id	✓	-	-	-	-
POST	/api/ai/providers/:id/test	✓	-	-	-	✓
GET	/api/monitor/kpis	✓	✓	-	-	✓
GET	/api/monitor/tokens	✓	-	-	-	✓
GET	/api/monitor/errors	✓	-	-	-	✓

✓* = solo datos de sus cursos/matriculas
 ✓** = solo sus propias entregas
 ✓***= solo tareas de sus cursos

6. FRONTEND: IMPLEMENTACIÓN (React)

6.1 Nuevas dependencias

```
pnpm add @tanstack/react-query # ya existe
# No se necesitan dependencias extra – todo vanilla React
```

6.2 Estructura de archivos nueva

```
src/
├── lib/
│   ├── api.ts           # (existe) – añadir interceptors
│   └── auth.tsx         # NUEVO – contexto de autenticación
├── components/
│   ├── ProtectedRoute.tsx # NUEVO – guard de rutas
│   └── LoginPage.tsx      # NUEVO – página de login
├── pages/
│   ├── Login.tsx         # NUEVO – wrapper de LoginPage
│   ├── Unauthorized.tsx  # NUEVO – página 403
│   └── ... (existentes)
└── App.tsx              # MODIFICAR – añadir rutas protegidas
```

6.3 Contexto de Autenticación (`src/lib/auth.tsx`)

```
import React, { createContext, useContext, useState, useEffect, useCallback } from 'react';

interface User {
```

```

    id: string;
    name: string;
    email: string;
    role: 'admin' | 'vicerrector' | 'docente' | 'estudiante' | 'sistemas';
    avatar_url?: string;
  }

interface AuthContextType {
  user: User | null;
  loading: boolean;
  login: (email: string, password: string) => Promise<void>;
  logout: () => Promise<void>;
  isAuthenticated: boolean;
  hasRole: (...roles: string[]) => boolean;
}

const AuthContext = createContext<AuthContextType>(null!);

export function AuthProvider({ children }: { children: React.ReactNode }) {
  const [user, setUser] = useState<User | null>(null);
  const [loading, setLoading] = useState(true);

  // Verificar sesión al montar
  useEffect(() => {
    fetch('/api/auth/me')
      .then(r => r.ok ? r.json() : null)
      .then(data => setUser(data))
      .catch(() => setUser(null))
      .finally(() => setLoading(false));
  }, []);

  const login = useCallback(async (email: string, password: string) => {
    const res = await fetch('/api/auth/login', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ email, password }),
      credentials: 'include'
    });

    if (!res.ok) {
      const err = await res.json();
      throw new Error(err.error || 'Error de autenticación');
    }

    const data = await res.json();
    setUser(data.user);
  }, []);

  const logout = useCallback(async () => {
    await fetch('/api/auth/logout', { method: 'POST', credentials: 'include' });
    setUser(null);
  }, []);

```

```

    }, []);

    const hasRole = useCallback((...roles: string[]) => {
      return user ? roles.includes(user.role) : false;
    }, [user]);

    return (
      <AuthContext.Provider value={{
        user, loading, login, logout,
        isAuthenticated: !!user,
        hasRole
      }}>
        {children}
      </AuthContext.Provider>
    );
  }
}

export const useAuth = () => useContext(AuthContext);

// Auto-refresh cada 10 minutos
export function useTokenRefresh() {
  const { isAuthenticated } = useAuth();

  useEffect(() => {
    if (!isAuthenticated) return;

    const interval = setInterval(async () => {
      try {
        await fetch('/api/auth/refresh', { method: 'POST', credentials: 'include' });
      } catch(e) {
        // Silencioso – si falla, el próximo request 401 redirige al login
      }
    }, 10 * 60 * 1000); // Cada 10 min

    return () => clearInterval(interval);
  }, [isAuthenticated]);
}

```

6.4 Componente ProtectedRoute ([src/components/ProtectedRoute.tsx](#))

```

import { Navigate, useLocation } from 'react-router-dom';
import { useAuth } from '@lib/auth';
import { Loader2 } from 'lucide-react';

interface Props {
  children: React.ReactNode;
  roles?: string[]; // roles permitidos, undefined = cualquier autenticado
}

```

```

export default function ProtectedRoute({ children, roles }: Props) {
  const { isAuthenticated, user, loading } = useAuth();
  const location = useLocation();

  if (loading) {
    return (
      <div className="flex h-screen items-center justify-center">
        <Loader2 className="size-8 animate-spin text-orange-500" />
      </div>
    );
  }

  if (!isAuthenticated) {
    return <Navigate to="/login" state={{ from: location }} replace />;
  }

  if (roles && user && !roles.includes(user.role)) {
    return <Navigate to="/no-autorizado" replace />;
  }

  return <>{children}</>;
}

```

6.5 Página de Login (`src/pages/Login.tsx`)

```

// Diseño responsivo con el tema Oxford (púrpura/naranja)
// - Formulario email + password
// - Validación en tiempo real
// - Mensajes de error del backend
// - Bloqueo visual después de 5 intentos
// - Link "¿Olvidaste tu contraseña?" (futuro)
// - Redirección post-login según rol:
//   admin → /sistemas
//   vicerrector → /vicerrectorado
//   docente → /docente
//   estudiante → /estudiante
//   sistemas → /sistemas

```

6.6 App.tsx Modificado

```

import { AuthProvider, useTokenRefresh } from '@lib/auth';
import Login from '@pages/Login';
import Unauthorized from '@pages/Unauthorized';
import ProtectedRoute from '@components/ProtectedRoute';

function AppRoutes() {
  useTokenRefresh();

```

```

return (
  <Routes>
    {/* Público */}
    <Route path="/" element={<Index />} />
    <Route path="/login" element={<Login />} />
    <Route path="/no-autorizado" element={<Unauthorized />} />

    {/* Docente */}
    <Route path="/docente" element={
      <ProtectedRoute roles={['docente', 'admin']}>
        <TeacherLayout />
      </ProtectedRoute>
    }>
      <Route index element={<TeacherDashboard />} />
      <Route path=":courseId" element={<TeacherCourse />} />
      <Route path=":courseId/tareas/:assignmentId" element={<TeacherAssignment />} />
    </Route>

    {/* Vicerrector */}
    <Route path="/vicerrectorado" element={
      <ProtectedRoute roles={['vicerrector', 'admin']}>
        <ViceLayout />
      </ProtectedRoute>
    }>
      <Route index element={<ViceDashboard />} />
    </Route>

    {/* Estudiante (NUEVO) */}
    <Route path="/estudiante" element={
      <ProtectedRoute roles={['estudiante']}>
        <StudentLayout />
      </ProtectedRoute>
    }>
      <Route index element={<StudentDashboard />} />
      <Route path="entrega/:id" element={<StudentSubmission />} />
    </Route>

    {/* Sistemas */}
    <Route path="/sistemas" element={
      <ProtectedRoute roles={['sistemas', 'admin']}>
        <SystemLayout />
      </ProtectedRoute>
    }>
      <Route index element={<SystemLogs />} />
      <Route path="ia" element={<AIConfig />} />
    </Route>

    <Route path="*" element={<NotFound />} />
  </Routes>
);

```

```

}

export default function App() {
  return (
    <QueryClientProvider client={queryClient}>
      <AuthProvider>
        <TooltipProvider>
          <Toaster />
          <Sonner />
          <BrowserRouter>
            <AppRoutes />
          </BrowserRouter>
        </TooltipProvider>
      </AuthProvider>
    </QueryClientProvider>
  );
}

```

6.7 Sidebar/Layout con menú según rol

Cada Layout existente (TeacherLayout, ViceLayout, SystemLayout) debe añadir:

```

// En el header de cada layout:
import { useAuth } from '@lib/auth';
import { Logout, User } from 'lucide-react';

function LayoutHeader() {
  const { user, logout } = useAuth();

  return (
    <header className="...">
      {/* ... contenido existente ... */}
      <div className="flex items-center gap-4">
        <span className="text-sm text-gray-400">
          {user?.name} · <span className="capitalize">{user?.role}</span>
        </span>
        <Button variant="ghost" size="sm" onClick={logout}>
          <Logout className="size-4 mr-1" /> Salir
        </Button>
      </div>
    </header>
  );
}

```

6.8 Landing Page ([Index.tsx](#)) — Adaptar

```
// Si el usuario YA está autenticado, mostrar links a su área
// Si NO, mostrar los botones actuales que llevan a /login

import { useAuth } from '@lib/auth';

function Index() {
  const { isAuthenticated, user } = useAuth();

  // En los botones CTA:
  {isAuthenticated ? (
    <Button asChild>
      <a href={getDashboardForRole(user!.role)}>
        Ir a mi Panel
      </a>
    </Button>
  ) : (
    <Button asChild>
      <a href="/login">Acceder</a>
    </Button>
  )}
}

function getDashboardForRole(role: string): string {
  const map: Record<string, string> = {
    admin: '/sistemas',
    vicerrector: '/vicerrectorado',
    docente: '/docente',
    estudiante: '/estudiante',
    sistemas: '/sistemas',
  };
  return map[role] || '/docente';
}
```

7. SEGURIDAD ADICIONAL

7.1 Protección contra ataques

Ataque	Defensa
Brute force	Lockout tras 5 intentos fallidos (30 min)
XSS	httpOnly cookies (JS no puede leer el token)

Ataque	Defensa
CSRF	SameSite=Lax + validación de Origin header
Session hijacking	Refresh token rotation (revocar anterior al rotar)
Password spraying	Rate limiting por IP (usando Map en memoria)
Timing attacks	bcrypt.compareSync (constant-time)

7.2 Rate Limiting (en memoria)

```
const rateLimitMap = new Map(); // IP → {count, resetAt}

function rateLimit(req, maxRequests = 20, windowMs = 60000) {
  const ip = req.headers['x-forwarded-for'] || req.socket.remoteAddress || 'unknown';
  const now = Date.now();

  let entry = rateLimitMap.get(ip);
  if (!entry || now > entry.resetAt) {
    entry = { count: 0, resetAt: now + windowMs };
    rateLimitMap.set(ip, entry);
  }

  entry.count++;
  return entry.count <= maxRequests;
}

// Limpiar el Map cada 5 minutos (evitar memory leak)
setInterval(() => {
  const now = Date.now();
  for (const [ip, entry] of rateLimitMap) {
    if (now > entry.resetAt) rateLimitMap.delete(ip);
  }
}, 5 * 60 * 1000);
```

7.3 Password Policy

- Mínimo 8 caracteres
- Al menos 1 mayúscula
- Al menos 1 número
- Al menos 1 carácter especial
- No puede contener el nombre de usuario ni email
- Historial de 3 contraseñas anteriores (no reutilizar)

8. PLAN DE IMPLEMENTACIÓN (FASES)

Fase 1: Backend — Core Auth (2-3 horas)

1. Instalar dependencias (`bcryptjs` , `jsonwebtoken`)
2. Crear tablas: columnas nuevas en users, refresh_tokens, audit_log
3. Implementar helpers: hash, verify, JWT sign/verify
4. Implementar endpoints: login, refresh, logout, me, password
5. Implementar middleware: requireAuth, requireRole
6. Hashear passwords de usuarios seed

Fase 2: Backend — Proteger API (2 horas)

1. Proteger cada endpoint según matriz de permisos
2. Añadir filtros por rol (docente ve sus cursos, estudiante sus entregas)
3. Añadir auditoría en operaciones sensibles

Fase 3: Frontend — Auth Context (1-2 horas)

1. Crear AuthContext + AuthProvider
2. Crear ProtectedRoute
3. Crear página de Login con diseño Oxford
4. Crear página Unauthorized (403)

Fase 4: Frontend — Migrar Layouts (1-2 horas)

1. Modificar App.tsx con rutas protegidas
2. Añadir user info + logout a cada Layout
3. Crear StudentLayout + StudentDashboard (nuevo)
4. Adaptar landing page

Fase 5: Hardening (1 hora)

1. Rate limiting para /api/auth/login
 2. Validación de password policy
 3. Tests de seguridad
-

9. ARQUITECTURA FINAL



```
refresh_tokens: token_hash, revoked
audit_log: user_id, action, entity, ip
```

10. PRÓXIMOS PASOS

Ejecutar en este orden cuando des la orden:

1. **Migrar BD** — Añadir columnas y tablas nuevas
2. **Implementar auth en serve.cjs** — Endpoints + middleware
3. **Proteger API existente** — requireAuth/requireRole en cada ruta
4. **Crear AuthProvider en React** — Contexto + hooks
5. **Crear Login page** — Diseño Oxford, formulario, validación
6. **Proteger rutas frontend** — ProtectedRoute + role guards
7. **Crear panel estudiante** — Nuevo layout y dashboard
8. **Probar** — Login/logout por cada rol, acceso denegado a rutas no permitidas